I confirm that the source code and the report content are my own work.

E-Signature:

Why was Shared memory used over global memory?

I used local histograms on shared memory. Each block creates a shared memory array:

```
__shared__ unsigned int local_hist[512];
```

The thread updates this local histogram before combining the results into global memory.

I have chosen shared memory as its faster than global memory due to it being located on-chip inside each streaming multiprocessor. If every thread updated the histogram directly via global memory, up to 2^24 atomic operations can occur, causing heavy contention and high memory latency. Instead, with local histograms, most updates happen in fast shared memory and you get at most `grid_size*512` number of atomic operations occurring.

Performance is improved by reducing the global memory traffic and lowering the number of global atomic operations. It also allows streaming multiprocessors to execute warps more efficiently with fewer memory stalls. However, some contention can still occur when many threads try to update the same shared-memory bin simultaneously.

Why were atomic operations used?

Atomic operations are required because many threads could try to update the same bucket simultaneously. Without them, two or more threads could read the same value together and overwrite each other's values. This would cause incorrect histogram values to be produced.

In my code, I used atomic operations when updating the shared-memory histogram:

```
atomicAdd( &(local_hist[bin]), 1);
```

This makes sure updates inside a block (e.g. incrementing bin) happen one at a time so every value is counted.

I also used atomicAdd when copying the local histogram into memory:

```
atomicAdd( &(local_hist[bin]), 1);
```

This prevents conflicts between different thread blocks and guarantees the output histogram is correct. This has to be done as thread blocks can be run in parallel on different streaming multiprocessors, which means that several blocks may try to update the same bin at once.

Table of Kernel, Call Number, Duration, Compute Throughput and Memory Throughput

Function Name	Call	Duration [ms]	Compute Throughput [%]	Memory Throughput [%]
maxReduceKernel	1	1.65	59.5	59.5
maxReduceKernel	2	0.01	11.98	11.98
maxReduceKernel	3	0.01	0.77	0.77
minReduceKernel	1	1.65	59.51	59.51
minReduceKernel	2	0.01	12.07	12.07
minReduceKernel	3	0.01	0.78	0.78
histogramKernel512	1	0.76	31.2	24.58

What kernel had the longest runtime?

The kernel calls that take the longest to run are the first calls for both the maxReduceKernal and minReduceKernal at 1.65ms. These also have the highest compute throughput and memory throughput values. This is because reduction operations continuously read large sections of global memory and need many parallel comparisons, meaning the kernels are able to keep a large number of threads active. This suggests that the kernels were using the Streaming Multiprocessors and memory bandwidth relatively efficiently.